

Rust Practical Projects and Data

Grounded in ECM2433 Rust project-structure, testing, slices/strings/data, HashMap, command-line parsing, and practical worksheet data-file material.

What You Need To Know

1. Project structure: *lib.rs*, *main.rs*, *crate names*, and *pub*

`src/lib.rs` is the library crate: put reusable logic here so it can be called from `main.rs` and tested. `src/main.rs` is the binary crate: it is the program entry point and contains `fn main()`. A good `main` usually collects input, calls library functions, then prints output.

The crate name used in use normally comes from the `[package]` name in `Cargo.toml`. Items are private by default. Mark functions, structs, fields, or modules with `pub` when code outside that module needs to use them.

```
my_project/
Cargo.toml      # [package] name = "my_project"
src/
  lib.rs        # reusable library logic
  main.rs       # binary entry point
tests/
  cli_tests.rs  # integration tests

/* src/lib.rs */
pub struct Item {
  pub name: String,
  pub quantity: u32,
  pub unit_price: f64,
}

pub fn line_total(item: &Item) -> f64 {
  item.quantity as f64 * item.unit_price
}

fn round_money(value: f64) -> f64 {
  (value * 100.0).round() / 100.0
}

/* src/main.rs */
use my_project::{line_total, Item};

fn main() {
  let item = Item {
    name: String::from("tea"),
    quantity: 3,
    unit_price: 1.25,
  };
  println!("{}", line_total(&item));
}
```

- `line_total` is public, so `main.rs` and integration tests can call it.
- `round_money` is private, so it can still be tested by unit tests inside `lib.rs`, but not called from `tests/`.
- `use` imports a path into scope. It does not copy text like C's `#include`.

2. Unit tests and integration tests

Unit tests usually sit beside the code in the same file. The usual shape is `#[cfg(test)] mod tests`. Inside that module, use `super::*`; imports the parent module, so unit tests can access private helpers.

Integration tests live in `tests/`. Each file there is compiled as a separate crate, like outside code using your library. That means integration tests can only use public API; they cannot use `super` to reach private helpers.

```
/* src/lib.rs */
pub fn double(n: i32) -> i32 {
  hidden_double(n)
}

fn hidden_double(n: i32) -> i32 {
  n * 2
}

#[cfg(test)]
mod tests {
  use super::*;

  #[test]
  fn private_helper_works() {
    assert_eq!(hidden_double(4), 8);
  }
}
```

```

/* tests/basic.rs */
use my_project::double;

#[test]
fn public_api_works() {
    assert_eq!(double(4), 8);
}

```

Tests can also return `Result`. That lets the test use `?` instead of `unwrap`.

```

#[test]
fn parses_number() -> Result<(), std::num::ParseIntError> {
    let n: i32 = "42".parse()?;
    assert_eq!(n, 42);
    Ok(())
}

```

3. Command-line arguments and parsing

`std::env::args()` gives command-line arguments as `String` values. They are text first, even if they look like numbers. Convert them with `parse`, then handle the `Result` with `match`, `expect`, or `?`.

```

/* cargo run -- 12 2.5 */
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();

    let program = &args[0]; // target/debug/my_project or similar
    let count_text = &args[1]; // "12"
    let scale_text = &args[2]; // "2.5"

    let count: i32 = match count_text.parse() {
        Ok(n) => n,
        Err(_) => {
            eprintln!("usage: {program} count scale");
            return;
        }
    };

    let scale: f64 = scale_text.parse().expect("scale must be a number");
    println!("{}", count as f64 * scale);
}

```

A cleaner practical pattern is to put parsing in a library function that returns `Result`, then keep `main` as a small caller.

```

/* src/lib.rs */
pub fn parse_count(text: &str) -> Result<i32, std::num::ParseIntError> {
    let n = text.trim().parse::<i32>()?;
    Ok(n)
}

/* src/main.rs */
fn main() {
    let arg = std::env::args().nth(1).expect("missing count");
    let count = my_project::parse_count(&arg).expect("bad count");
    println!("{}", count);
}

```

4. Reading data files and parsing records

Practical projects often read text files with `std::fs::read_to_string`. That gives one `String`. Then split it into lines, parse each line, and store structured data in a struct.

```

#[derive(Debug, Clone)]
pub struct Item {
    pub name: String,
    pub quantity: u32,
    pub unit_price: f64,
}

pub fn parse_item(line: &str) -> Result<Item, String> {
    let parts: Vec<&str> = line.split(',').collect();
    if parts.len() != 3 {
        return Err("expected name,quantity,unit_price".to_string());
    }

    let quantity = parts[1]
        .trim()
        .parse::<u32>()
        .map_err(|_| "bad quantity".to_string())?;

    let unit_price = parts[2]
        .trim()
        .parse::<f64>()
        .map_err(|_| "bad price".to_string())?;

    Ok(Item {
        name: parts[0].trim().to_string(),
    })
}

```

```

        quantity,
        unit_price,
    })
}

pub fn load_items(path: &str) -> Result<Vec<Item>, String> {
    let text = std::fs::read_to_string(path)
        .map_err(|err| err.to_string())?;

    let mut items = Vec::new();
    for line in text.lines() {
        if !line.trim().is_empty() {
            items.push(parse_item(line)?);
        }
    }
    Ok(items)
}

```

5. HashMap aggregation

HashMap<K, V> stores key-value data. Use get when looking up a value: it returns Option because the key might not exist. Use entry(...).or_insert(...) when counting or accumulating because it handles the missing-key case and gives a mutable value.

```

use std::collections::HashMap;

pub fn print_score(scores: &HashMap<String, u32>, team: &str) {
    match scores.get(team) {
        Some(score) => println!("{team}: {score}"),
        None => println!("{team} has no score"),
    }
}

pub fn count_words(words: &[&str]) -> HashMap<String, u32> {
    let mut counts = HashMap::new();

    for word in words {
        let key = word.to_lowercase();
        *counts.entry(key).or_insert(0) += 1;
    }

    counts
}

pub fn total_by_name(items: &[Item]) -> HashMap<String, f64> {
    let mut totals = HashMap::new();

    for item in items {
        let total = item.quantity as f64 * item.unit_price;
        *totals.entry(item.name.clone()).or_insert(0.0) += total;
    }

    totals
}

```

The star in `*map.entry(...).or_insert(...)` is there because `or_insert` returns a mutable reference to the value in the map. Dereferencing lets you update the stored value.

6. Slices, iterators, enumerate, and chunks

Use slices when a function only needs to read a sequence. `&[T]` borrows a view of contiguous data, so the same function can accept a whole `Vec<T>`, an array slice, or a chunk. Iterating with `enumerate` gives both the index and a borrowed value.

```

pub fn first_greater_than(values: &[i32], threshold: i32) -> Option<usize> {
    for (index, value) in values.iter().enumerate() {
        if *value > threshold {
            return Some(index);
        }
    }
    None
}

pub fn batch_totals(values: &[i32], chunk_size: usize) -> Vec<i32> {
    values
        .chunks(chunk_size)
        .map(|chunk| chunk.iter().sum())
        .collect()
}

```

Practice Questions

Question 1 - fix the project visibility:

```

/* src/lib.rs */
fn triple(n: i32) -> i32 { n * 3 }

```

```

/* src/main.rs */
use calc::triple;

fn main() {
    println!("{}", triple(4));
}

```

- 1 Explain why this fails, then fix the library function.

Question 2 - write the unit test:

```

pub fn square(n: i32) -> i32 { n * n }

/* write the test module here */

```

- 2 Write a unit test module that checks `square(5)`.

Question 3 - integration test access:

```

/* src/lib.rs */
fn helper(n: i32) -> i32 { n + 1 }
pub fn public_api(n: i32) -> i32 { helper(n) }

/* tests/basic.rs */
use calc::helper;

```

- 3 Explain why the integration test cannot import `helper`, then give the normal fix.
- 4 For `cargo run -- 12 2.5`, say what usually appears in `args[0]`, `args[1]`, and `args[2]`.

Question 5 - replace the unsafe parse:

```

let count: i32 = args[1].parse().unwrap();

```

- 5 Rewrite this with `match` so invalid input prints an error and returns.

Question 6 - complete the parsing helper:

```

pub fn parse_count(text: &str) -> Result<i32, std::num::ParseIntError> {
    let n = /* parse text as i32 using ? */;
    Ok(n)
}

```

- 6 Complete the missing expression.

Question 7 - complete the HashMap counter update:

```

use std::collections::HashMap;

let mut counts: HashMap<String, u32> = HashMap::new();
let name = String::from("Ada");

/* increment the count for name */

```

- 7 Write the `entry` API line that increments the count for `name`.

Question 8 - complete the accumulating totals update:

```

struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

let mut totals: HashMap<String, f64> = HashMap::new();
let item = Item {
    name: String::from("tea"),
    quantity: 3,
    unit_price: 1.25,
};

/* add this line total to any existing total for the same product name */

```

- 8 Write the update line so repeated items with the same name are accumulated, not replaced or ignored.

Question 9 - write the slice scan:

```

pub fn first_greater_than(values: &[i32], threshold: i32) -> Option<usize> {
    /* complete this */
}

```

- 9 Complete the function so it returns the index of the first value greater than the threshold, or `None`.

Question 10 - destructuring and references:

```

let rows = vec![
    (String::from("red"), 10),
    (String::from("blue"), 7),
];

for (name, score) in rows.iter() {

```

```
        println!("{name}: {score}");  
    }
```

- 10** Explain what `(name, score)` does here and why `score` is a reference.

Mark Scheme

Attempt all questions before checking.

- 1 Functions are private by default. Fix: `pub fn triple(n: i32) -> i32 { n * 3 }`.
- 2 Expected shape: `#[cfg(test)] mod tests { use super::*; #[test] fn square_works() { assert_eq!(square(5), 25); } }`.
- 3 Integration tests are separate crates and can only use public API. Normal fixes: test through `public_api`, or make the helper public if it is genuinely part of the public API.
- 4 `args[0]` is the program path/name. `args[1]` is 12. `args[2]` is 2.5. They are all strings.
- 5 Expected pattern: `let count: i32 = match args[1].parse() { Ok(n) => n, Err(_) => { eprintln!("invalid count"); return; } }`;
- 6 `let n = text.trim().parse::<i32>()?;`. The function returns `Result`, so `?` can return parse errors early.
- 7 `*counts.entry(name).or_insert(0) += 1;`
- 8 `*totals.entry(item.name.clone()).or_insert(0.0) += item.quantity as f64 * item.unit_price;`. The starting value is 0.0 only for a missing key; existing totals are increased.
- 9 Loop with `values.iter().enumerate()`. If `*value > threshold`, return `Some(index)`. After the loop, return `None`.
- 10 Tuple destructuring splits each pair into its two parts. Because `rows.iter()` borrows `rows`, `name` and `score` are references to the tuple fields rather than owned values.

Useful Code Patterns

```
/* Unit test */
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn works() {
        assert_eq!(double(4), 8);
    }
}

/* Integration test: tests/basic.rs */
use my_project::double;

#[test]
fn public_api_works() {
    assert_eq!(double(4), 8);
}

/* Parse with match */
let n: i32 = match text.parse() {
    Ok(value) => value,
    Err(_) => {
        eprintln!("invalid number");
        return;
    }
};

/* HashMap counting */
*counts.entry(key).or_insert(0) += 1;

/* Slice scan */
for (index, value) in values.iter().enumerate() {
    if *value > threshold {
        return Some(index);
    }
}
None
```